Lecture Notes: Neural Network Part I

Course: 2110573 Pattern Recognition

2026-02-15
version 1.0

1 Introduction to Machine Learning Models

Machine learning models can be categorized in several ways, based on how they learn, what they output, and their underlying modeling philosophy.

1.1 Learning Paradigms

- **Supervised Learning:** The model learns from labeled data, where each data point has a known outcome or target.
- **Unsupervised Learning:** The model learns from unlabeled data, identifying patterns or structures within the data itself.
- **Reinforcement Learning:** The model learns by interacting with an environment, receiving rewards or penalties for its actions.

1.2 Output Types

- **Regression:** The model predicts a continuous value (e.g., predicting a house price).
- **Classification/Clustering:** The model predicts a discrete category or label (e.g., classifying an email as spam or not).

1.3 Modeling Philosophy: Generative vs. Discriminative

This distinction is fundamental to understanding different types of classification models.

1.3.1 Generative Models

Generative models learn the underlying distribution of the data for each class. They model the **likelihood** $P(\mathbf{x}|y)$, which is the probability of observing the input features $\mathbf{x}$ given a class y.

- Examples include Naive Bayes and Bayesian classifiers.
- To perform classification, they use Bayes' rule to find the class y that maximizes the **posterior probability** $P(y|\mathbf{x})$:
$$y^* = \arg\max_y P(y|\mathbf{x}) = \arg\max_y \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})}$$
- In this equation, $P(y)$ is the **prior** probability of the class, and $P(\mathbf{x})$ is the evidence. Since $P(\mathbf{x})$ is constant for all classes, it can be ignored during maximization.
- An alternative view is that generative models learn the joint probability distribution $P(\mathbf{x}, y)$, since $P(\mathbf{x}, y) = P(\mathbf{x}|y)P(y)$.
- A key advantage is that they can be used to **generate** new data samples $\mathbf{x}$ by sampling from the learned likelihood distribution $P(\mathbf{x}|y)$.

1.3.2 Discriminative Models

Discriminative models learn the decision boundary between classes directly. They model the **posterior probability** $P(y|\mathbf{x})$ without learning about the underlying data distribution.

- Examples include Logistic Regression, Support Vector Machines (SVMs), and Neural Networks.
- The goal is to directly find a function that maps inputs $\mathbf{x}$ to a class label y:

$$y^* = \arg\max_y P(y|\mathbf{x})$$

- In general, this can be any function $h(\mathbf{x}, y)$ that gives a score for each class.
- **Advantages:** Usually achieve higher classification performance because they focus directly on the discrimination task.
- **Disadvantages:** They cannot be used to generate new data. Adding a new class requires retraining the entire model from scratch.

2 Neural Networks (Deep Learning)

Deep Learning, Deep Neural Networks (DNNs), and Neural Networks are largely synonymous terms. They are a class of discriminative models that have become state-of-the-art in many fields.

2.1 Why Now?

Although the concepts have existed since the 1990s, the recent explosion in their popularity is due to a confluence of factors:

- **Big Data:** DNNs are exceptionally good at leveraging massive datasets to learn complex patterns, outperforming other models when data is abundant.
- **GPU (Hardware):** The parallel architecture of Graphics Processing Units (GPUs) is perfectly suited for the matrix and vector operations that are central to neural networks, making it feasible to train much larger and deeper models.
- **Deep Architectures:** Researchers have found that very deep models, contrary to earlier belief, are often easier to optimize. The high-dimensional loss landscapes of deep networks have fewer "bad" local minima than shallower ones. This is connected to the "double descent" phenomenon, where performance first improves, then worsens (overfitting), and then improves again as model complexity (size/depth) passes a certain threshold.

2.2 Inductive Bias

The **inductive bias** of a learning algorithm is the set of assumptions it makes to generalize from the training data to new, unseen inputs.

- For example, a linear model assumes a linear relationship between features and the output.
- In traditional machine learning, inductive bias is often introduced through manual **feature engineering**.
- In deep learning, inductive bias is encoded in the network's **architecture** (e.g., using convolutional layers for images or recurrent layers for sequences). This allows the model to learn relevant features automatically.

3 Fully Connected Networks (FCN)

Also known as Feed-Forward Networks or Multilayer Perceptrons (MLPs), these are the most basic type of neural network.

3.1 The Neuron

An artificial neuron is a computational unit inspired by biological neurons. It takes multiple weighted inputs, sums them up with a bias term, and then passes the result through a non-linear **activation function**.

$$\text{output} = f\left(\sum_i w_i x_i + b\right)$$

where w_i are the weights, x_i are the inputs, b is the bias, and f is the activation function.

3.2 Layers and Architecture

- **Input Layer:** Receives the raw input data (e.g., pixels of an image, words in a sentence). It's good practice to scale inputs to have zero mean and unit variance.
- **Hidden Layers:** A series of layers between the input and output. Each layer is composed of multiple neurons. The "deep" in deep learning refers to having many hidden layers.
- **Output Layer:** The final layer that produces the model's prediction. The structure of this layer depends on the task.

A single neuron creates a single hyperplane as a decision boundary. By stacking neurons in layers, the network can combine these hyperplanes to create arbitrarily complex, non-linear decision boundaries. From a linear algebra perspective, a neural network layer applies a linear transformation (matrix multiplication by weights) followed by a non-linear transformation (the activation function).

$$\mathbf{h} = f(\mathbf{W}^T\mathbf{x} + \mathbf{b})$$

3.3 The Importance of Non-linearity

The activation function introduces non-linearity. Without it, a stack of multiple linear layers would be mathematically equivalent to a single linear layer, because a series of matrix multiplications can be combined into one.

$$\mathbf{W_2}(\mathbf{W_1}\mathbf{x}) = (\mathbf{W_2}\mathbf{W_1})\mathbf{x} = \mathbf{W_{eff}}\mathbf{x}$$

The non-linearity is what allows the network to learn complex, non-linear functions.

3.3.1 Common Activation Functions

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$. Squashes values to the range (0, 1). Suffers from the vanishing gradient problem.
- **Tanh (Hyperbolic Tangent):** $\tanh(x)$. Squashes values to (-1, 1). It is zero-centered, which is an advantage over sigmoid, but also suffers from vanishing gradients.
- **ReLU (Rectified Linear Unit) [12]:** $f(x) = \max(0, x)$. Very popular and computationally efficient. It does not saturate in the positive direction but can "die" (output zero) for all negative inputs.
- **Leaky ReLU / PReLU [11][4]:** A variant of ReLU that allows a small, non-zero gradient for negative inputs ($f(x) = \max(0.01x, x)$), preventing the "dying ReLU" problem. In PReLU, the slope for negative values is a learnable parameter.
- **Swish/SiLU (Sigmoid Linear Unit) [13][3]:** $f(x) = x \cdot \sigma(\beta x)$. A smooth, non-monotonic function that often performs better than ReLU.
- **GELU (Gaussian Error Linear Unit) [5]:** $f(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the Gaussian cumulative distribution function. Widely used in modern transformer models.
- **GLU (Gated Linear Unit) [2]:** A mechanism that uses a gating function, often sigmoid, to control the information flow. The SwiGLU [14] variant, which uses Swish for gating, is very effective in large language models.

Practical advice: Start with ReLU. If performance is an issue, try Leaky ReLU or GELU/SwiGLU.

3.4 Softmax Output Layer

For multi-class classification, we want the output to be a probability distribution over the classes. The softmax function is used for this. It takes the raw output scores (logits) and converts them into probabilities that sum to 1.

$$P(y = j|\mathbf{x}) = \frac{e^{h_j}}{\sum_k e^{h_k}}$$

where h_j is the logit for class j.

4 Training Neural Networks

Training a neural network involves finding the optimal set of weights and biases (θ) that allows the model to map inputs to their correct outputs. This is framed as an optimization problem where we aim to minimize a predefined error or "loss" function.

4.1 Objective (Loss) Function

The loss function (also called the objective or cost function) quantifies how far the model's prediction is from the true target value. The choice of loss function is critical and depends entirely on the nature of the task (e.g., classification or regression). The overall loss for the dataset is typically the average of the per-sample losses.

4.1.1 Classification Losses

For classification, we want to measure the discrepancy between the predicted probability distribution and the true distribution (which is a one-hot vector indicating the correct class).

Cross-Entropy Loss (Log Loss) This is the most common loss function for classification tasks. It is derived from information theory and measures the "distance" between two probability distributions. Minimizing the cross-entropy loss is equivalent to maximizing the log-likelihood of the data given the model.

For a single data point, the multi-class cross-entropy loss is:

$$L(\mathbf{y}, \mathbf{q}) = -\sum_{c=1}^{C} y_c \log(q_c)$$

where:

- C is the number of classes.
- $\mathbf{y}$ is the true distribution, a one-hot vector where the correct class c' has $y_{c'} = 1$ and all other elements are 0.
- $\mathbf{q}$ is the model's predicted probability distribution, typically the output of a **softmax** layer, where q_c is the predicted probability for class c.

Since only one element in $\mathbf{y}$ is non-zero, the sum simplifies to just the negative log probability of the true class:

$$L = -\log(q_{c'})$$

The goal is to make the predicted probability for the correct class, $q_{c'}$, as close to 1 as possible, which in turn drives the loss L towards 0.

Binary Cross-Entropy (BCE) For a binary classification problem (two classes, 0 and 1), the formula simplifies. The output of the model is typically a single neuron with a **sigmoid** activation, which outputs a probability q. The BCE loss is:

$$L(y, q) = -[y \log(q) + (1 - y) \log(1 - q)]$$

where:

- y is the true label (0 or 1).
- q is the predicted probability of the class being 1.

This formula ensures that when the true label is 1, the loss is driven by $-\log(q)$, and when the true label is 0, the loss is driven by $-\log(1-q)$.

Squared Earth Mover's Distance (EMD) Loss [7] The Earth Mover's Distance (EMD), also known as the Wasserstein distance, is a loss function that measures the minimum cost required to transform one probability distribution into another. Unlike cross-entropy, which treats classes as independent and unordered, EMD explicitly accounts for the underlying structure or ordering of classes. This makes it particularly suitable for classification problems where class labels have a meaningful ordinal or spatial relationship.

For a single data point, let $\mathbf{y} \in \mathbb{R}^C$ be the ground-truth distribution and $\mathbf{q} \in \mathbb{R}^C$ be the predicted probability distribution, where C is the number of classes. The squared Earth Mover's Distance loss is defined as:

$$L_{\text{EMD}}(\mathbf{y}, \mathbf{q}) = \sum_{i=1}^{C} \left(\sum_{j=1}^{i} (q_j - y_j) \right)^2$$

where the inner summation represents the cumulative distribution difference up to class i.

In the case of classification, $\mathbf{y}$ is typically a one-hot vector indicating the true class. The squared EMD penalizes predictions proportionally to how far the predicted probability mass is from the true class, resulting in larger penalties for errors that are farther away in class order. This property allows squared EMD to capture semantic distances between classes that are ignored by standard cross-entropy loss.

4.1.2 Regression Losses

For regression, the goal is to predict a continuous value. The loss function measures the magnitude of the difference between the predicted value and the true value.

Mean Squared Error (MSE) or L2 Loss MSE is the most widely used regression loss. It calculates the average of the squared differences between the predicted and true values.

$$L(y, q) = \frac{1}{N} \sum_{n=1}^{N} (y_n - q_n)^2$$

- **Pros:** It is a strongly convex function with a clear minimum, and it penalizes larger errors more heavily due to the squaring operation. Its derivative is straightforward to compute.
- **Cons:** Its sensitivity to large errors also makes it sensitive to outliers. A single data point with a large error can dominate the gradient and skew the training process.

Mean Absolute Error (MAE) or L1 Loss MAE calculates the average of the absolute differences between the predicted and true values.

$$L(y, q) = \frac{1}{N} \sum_{n=1}^{N} |y_n - q_n|$$

- **Pros:** It is much more robust to outliers compared to MSE because the error is not squared.
- **Cons:** The gradient is constant for all errors (either +1 or -1), which can make convergence tricky, especially as the model gets closer to the minimum (the gradient doesn't shrink). The loss function is not differentiable at zero, though this is easily handled in practice.

Huber Loss (Smooth Mean Absolute Error) Huber Loss offers a compromise between MSE and MAE. It behaves like MSE for small errors and like MAE for large errors. It is defined by a threshold parameter, δ.

$$L_\delta(y, q) = \begin{cases} \frac{1}{2}(y-q)^2 & \text{for } |y-q| \leq \delta \\ \delta(|y-q| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

- It combines the best of both worlds: it is less sensitive to outliers than MSE, while still being differentiable at zero and having a stronger gradient than MAE for small errors, which helps with convergence.

4.2 Gradient Descent and Backpropagation

We minimize the loss function L with respect to the parameters θ using an iterative optimization algorithm called gradient descent.

1. **Forward Pass:** Input data is passed through the network to compute the output and the final loss.
2. **Backward Pass (Backpropagation):** The algorithm computes the gradient of the loss with respect to each parameter in the network, starting from the output layer and moving backward. This is a practical application of the chain rule of calculus. Modern deep learning frameworks automate this process via **autograd**.
3. **Parameter Update:** The parameters are updated in the opposite direction of their gradient, scaled by a **learning rate** α.
$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta L(\theta_t)$$

4.3 Stochastic Gradient Descent (SGD)

Computing the gradient over the entire dataset (batch gradient descent) is computationally expensive. Instead, **Stochastic Gradient Descent (SGD)** computes the gradient on a small, random subset of the data called a **mini-batch**.

- This is much faster and the noise from the mini-batch sampling can help the model escape local minima.
- **Epoch:** One full pass through the entire training dataset.

4.4 Optimization Algorithms

While standard SGD is effective, its convergence can be slow and sensitive to the learning rate. Several advanced optimizers have been developed to address these issues by incorporating more sophisticated update rules.

- **Momentum:** This method helps accelerate SGD in the relevant direction and dampens oscillations. It does this by adding a fraction β of the previous update vector to the current gradient step. This accumulated gradient is often interpreted as a "velocity" term.
$$v_{t+1} = \beta v_t + \alpha \nabla_\theta L(\theta_t)$$
$$\theta_{t+1} = \theta_t - v_{t+1}$$
Here, v_t is the velocity at time t, and β is the momentum coefficient (typically set to 0.9).
- **Nesterov Momentum (Nesterov Accelerated Gradient - NAG):** A smarter version of momentum. Instead of calculating the gradient at the current position, it first makes a "look-ahead" step in the direction of the accumulated velocity and calculates the gradient from there. This allows it to correct its course faster.
$$v_{t+1} = \beta v_t + \alpha \nabla_\theta L(\theta_t - \beta v_t)$$
$$\theta_{t+1} = \theta_t - v_{t+1}$$

- **Adaptive Learning Rates:** These methods dynamically adjust the learning rate for each parameter individually. The intuition is to perform larger updates for infrequent parameters and smaller updates for frequent ones.

- **RMSProp (Root Mean Square Propagation) [6]:** RMSProp maintains a moving average of the squared gradients for each parameter and divides the learning rate by the square root of this average.

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma)(\nabla_\theta L)^2$$
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} \nabla_\theta L$$

  where γ is a decay rate and ϵ is a small smoothing constant to prevent division by zero.

- **Adam (Adaptive Moment Estimation) [9]:** Adam is arguably the most popular and widely used optimizer. It combines the ideas of both momentum (first-moment estimation) and adaptive learning rates (second-moment estimation, like RMSProp). It computes adaptive learning rates for each parameter.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta L \quad \text{(Momentum)}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta L)^2 \quad \text{(RMSProp-like)}$$

  Bias-corrected estimates are then computed:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

  And the final update is:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

- **AdamW [10]:** A crucial modification to Adam that improves generalization. Standard L2 regularization in Adam is often implemented by adding the squared weights to the loss, which couples it with the gradient. AdamW decouples the weight decay from the gradient update. Instead of modifying the gradient, it applies the weight decay directly to the weights after the main Adam update step. This often results in better model performance.

- **Lion (EvoLved Sign Momentum) [1]:** A newer optimizer discovered through evolutionary algorithm search. It is simpler and more memory-efficient than Adam because it only tracks the momentum (first moment), not the second moment. The update is derived from the sign of the momentum.

$$\theta_{t+1} = \theta_t - \alpha \cdot \text{sign}(\beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta L)$$

  Despite its simplicity, it has been shown to outperform Adam on various tasks, especially in training large-scale models.

4.5 Learning Rate

The learning rate is arguably the most important hyperparameter to tune.

- **Too large:** The optimization may diverge.

- **Too small:** Training will be very slow and may get stuck in poor local minima.

- **Learning Rate Scheduling:** It's common practice to decrease the learning rate during training. Strategies include step decay, exponential decay, or using a "warm-up" phase where the learning rate starts small, increases, and then decays.

5 Regularization: Preventing Overfitting

Overfitting occurs when a model learns the training data too well, including its noise, and fails to generalize to new, unseen data. Regularization techniques are used to combat this.

5.1 Explicit Regularization

These methods add a penalty term to the loss function.

- **L2 Regularization:** Adds a penalty proportional to the sum of the squared weights. This encourages the model to use smaller, more diffuse weights.

$$L_{reg} = L_{orig} + \lambda \sum_j \theta_j^2$$

  This is the same as weight decay if the learning rate is fixed and no adaptive learning rate techniques, e.g. ADAM, are used.

- **L1 Regularization:** Adds a penalty proportional to the sum of the absolute values of the weights. This can lead to sparse weights (many weights becoming exactly zero), which can be useful for feature selection.

$$L_{reg} = L_{orig} + \lambda \sum_j |\theta_j|$$

5.2 Implicit Regularization

These methods introduce regularization through the training process or network architecture.

- **Early Stopping:** Monitor the model's performance on a separate **validation set** during training and stop when performance on this set stops improving, even if the training loss is still decreasing.

- **Dropout [15]:** During training, randomly set the output of some neurons to zero with a certain probability at each forward pass. This forces the network to learn redundant representations and prevents it from relying too much on any single neuron. At test time, all neurons are used, but their outputs are scaled down by the dropout probability.

- **Batch Normalization [8]:** Normalizes the output of a previous layer by subtracting the mini-batch mean and dividing by the mini-batch standard deviation. It is then rescaled and shifted by two learnable parameters. This stabilizes and accelerates training, and also acts as a regularizer. It is typically placed before the non-linearity.

6 Summary of Key Concepts

- **Layers:** The building blocks of networks (Dense, Softmax, Dropout, Batchnorm).

- **Non-linearity:** The crucial ingredient (ReLU, GELU, etc.) that allows learning complex functions.

- **Loss Function:** A measure of model error (Cross-Entropy for classification, MSE for regression) that we aim to minimize.

- **Optimizers:** Algorithms (SGD, Adam, AdamW) that update model weights to minimize the loss, guided by gradients computed via backpropagation.

- **Learning Rate:** The step size for the optimizer, a critical hyperparameter that often requires careful tuning and scheduling.

- **Regularization:** Techniques (L1/L2, Dropout, Batchnorm, Early Stopping) to prevent overfitting and improve generalization.

References

[1] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, et al. Symbolic discovery of optimization algorithms. *Advances in neural information processing systems*, 36:49205–49233, 2023.

[2] Yann N Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. In *International conference on machine learning*, pages 933–941. PMLR, 2017.

[3] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

[5] D Hendrycks. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[6] Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. *Cited on*, 14(8):2, 2012.

[7] Le Hou, Chen-Ping Yu, and Dimitris Samaras. Squared earth mover's distance-based loss for training deep neural networks. *CoRR*, abs/1611.05916, 2016.

[8] Sergey Ioffe. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.

[9] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[10] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[11] Andrew L Maas, Awni Y Hannun, Andrew Y Ng, et al. Rectifier nonlinearities improve neural network acoustic models. In *Proc. ICML*, volume 30, no. 1, page 3. Atlanta, GA, 2013.

[12] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.

[13] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

[14] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[15] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.